The Power of Precision: The Advantages and Limitations of Linear and Non-Linear Phase EQs

By Patrick Lacey

Parametric EQs have always been my first choice of EQs when producing or mixing because of it's flexibility to sculpt the frequency response. So as a long time Ableton Live user, I am creating an eight band parametric EQ plug-in for my project that is based off of the stock EQ Eight. Throughout the creation, I learned a lot about the phase shifting that happens within EQs, and was able to compare my plug-in's phase changes with multiple commercial stock and third-party EQs.

I am going to be using JUCE's framework and code this using C++ and Visual Code as the IDE. When planning out this project, I decided that I was going to use an object from JUCE's dsp class called ProcessorChain. This object allows me to call a "chain" of effects and makes the processing more efficient and easier to follow. For example, calling reset, prepare, or process functions on the chain performs the function for all the objects inside the chain. The chain that I decided to go with is eight EQs bookended by input and output gain control. To make it a little easier to keep track of which EQ I am working with, I created another ProcessorChain of four EQs that I call twice within my main ProcessorChain. To create an Infinite Impulse Response (IIR) EQ outside of a ProcessorChain, you would have to call a ProcessorDuplicator to make the effect stereo. So to process a stereo signal when using the ProcessorChain, I created two chains for the individual left and right channels.

Creating the parametric ability within the EQs entailed creating a parameter using AudioParameterChoice that lists the filter types. When one is selected, it creates the correct coefficients for the desired filter within my updateChain() function.

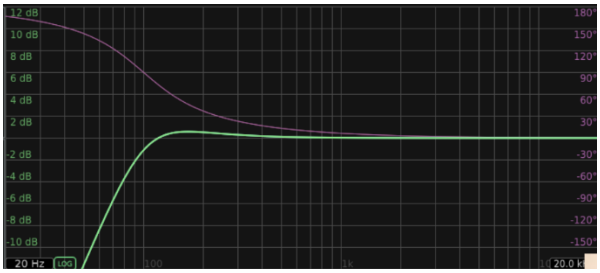
When choosing either a high pass or low pass filter, I wanted the ability of deciding the filter slope. One option is to call the version of the makeLowPass or makeHighPass functions that includes the Q value in the process of calculating the coefficients. This does make the filters slope adjustable, but it adds a resonance peak which gets more defined the higher the Q value. Another option is to chain together a couple EQs with the same low or high pass functions cutting off at the same frequency, which will make the slope steeper by 12dB/oct per chained EQ. The issue is that when making an EQ with eight parametric filters, there would be three to four times the amount of filters needed depending on the desired cutoff slope. So for the interest of CPU usage, I decided to go with the function call that uses the Q value, and have to accept the resonant bump it adds.

During my research, I found out that there is a filter object that is designed to have less audio artifacts when doing real time manipulation of the filter parameters called the StateVariableTPTFilter. This object only creates highpass, lowpass, and allpass filters though, so IIR filter objects would still have to be called for the peak, and low/high shelf filters. This causes a problem when processing these parametric filters using the ProcessorChain. So I've decided to stay using the IIR filters, and focus on smoothing the change in the parameters when needed.

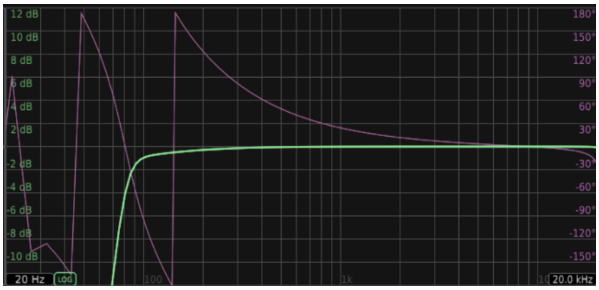
In doing research for this project, I came across the distinction between non-linear and linear phase EQs. Which in all honesty, the thought of phasing issues due to EQ processing had never came to mind, so I decided to look into it further. In theory, most EQs phase shift to emphasize or cancel out specific frequency bands, so it makes sense to see a shift in phase after EQ. Most non specified EQs are non-linear EQs, which alters the phase of the signal. To achieve a linear phase EQ, an Finite Impulse Response (FIR) filter would need to be used, which is way more CPU intensive and causes more latency than an IIR filter. Ultimately, I decided to continue to use IIR filters and accept the shift in phase in the filtered frequency bands because it's rarely noticable. In my opinion, one of the few necessary uses of a

linear phase EQ is when one is mixing a live drum kit or any situation where there are multiple mics on the same instrument. I say a drum kit because the transients are very important to maintain for drums.

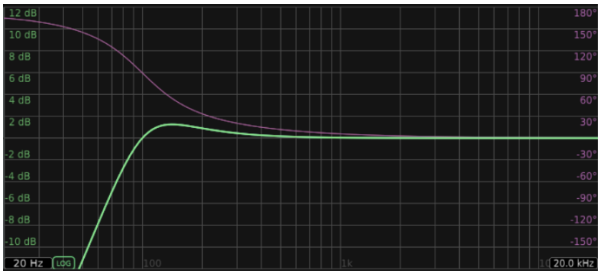
After making that decision, I still wanted to check the output phase of my plugin and compare it to other commercial plug-ins. I grabbed a couple of EQs from different companies, and used Bertom's EQ Curve Analyzer to diplay the phase of the signal. The EQ Curve Analyzer's x axis represents the frequency range (Hz), and the y axis represents the gain (dB) and the phase shift applied (degrees). I compared the phase shifts between plug-ins with every filter type, and I found that for most of the filter types, the phase shift is very similar between all plug-ins. I based my analysis on a highpass filter with a cutoff frequency at 100 Hz, because the there was the most difference in the phase shift between plug-ins using a high or low pass, and low end phasing issues are a common reason for using non-linear phase EQs.

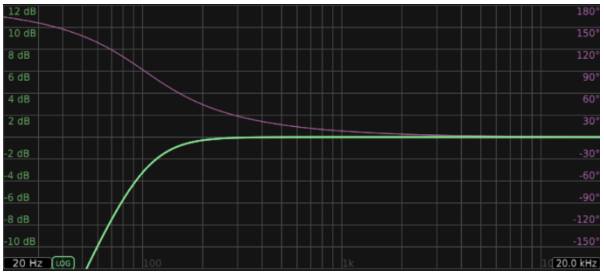


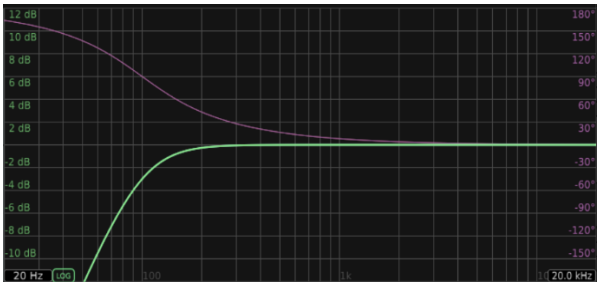
This is the response using the EQ that I created. The frequency response is in green and the phase is in purple. As you can see the phase close to an inverted curve of the frequency filter. I dailed in the Q value to about 0.75, so there is a steeper slope, but no significant resonant peak.

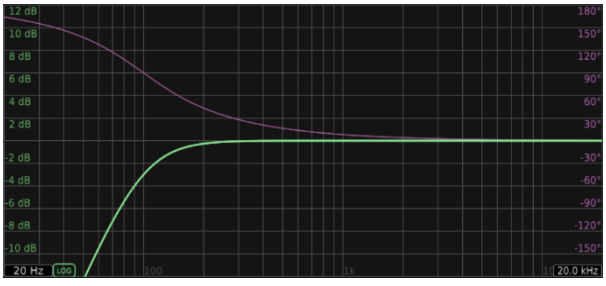


This is the response of Ableton Live's stock EQ Eight plug-in. This phase looks significantly different than my EQ's phase. It looks to have the some limit spikes in the effected low frequencies.

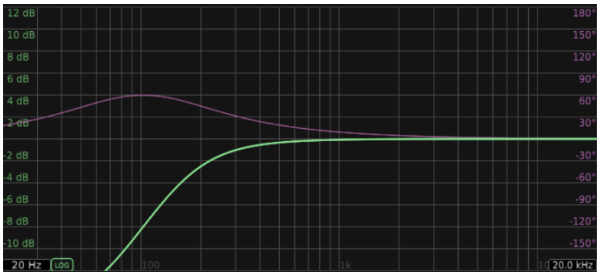




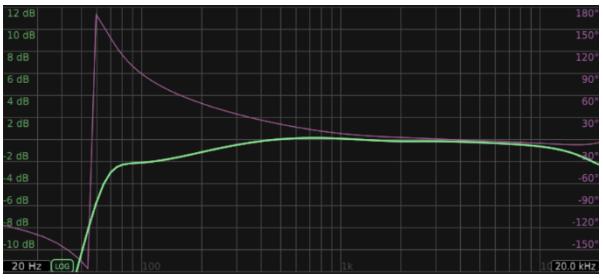


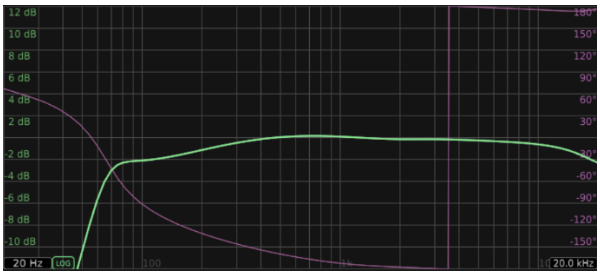


The pictures above are the responses to iZotope's Neutron 3, KiloHearts Filter, and Waves Q1 and AudioTrack. All of the phase responses look very similar to my plug-in. The frequency response of the Neutron 3 shows a slight resonant peak, which leads me to believe that it calculates the highpass coefficients regarding the Q value and therefore is probably the closest comparison that I have for the plug-in I created.

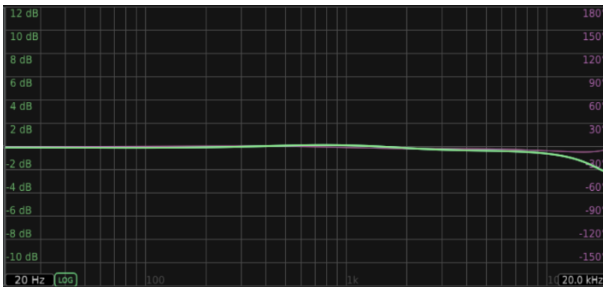


This is the KiloHearts 3 Band EQ's response. It has the least ammount of phase shifting out of all the plug-ins I tested. This is because the 3 band EQ's low frequency filter is a low shelf instead of a highpass, which effects the phase differently.

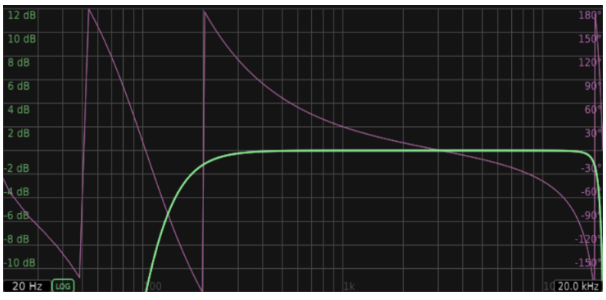




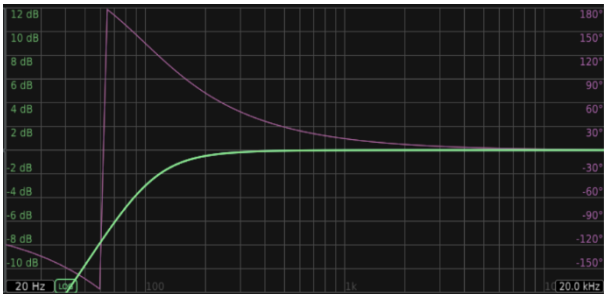
The two photos above are screenshots of the same plug-in's response, Waves VEQ3, but with it's phase flip button being toggled (left: OFF, right: ON). When the buttons off, the phase response looks somewhat similar to Ableton Live's EQ Eight, but the button seems to push the spike into higher frequencies and there still is phase shifting within the filtered low frequency range.



The frequency response has some initial filtering, which causes no phase shifting. My best guess is the minimal filtering doesn't shift the phase enough for a visible difference.



This is a response from another project of mine where I tried out using the StateVariableTPTFilter and the multiple chained filters method to get a steeper cutoff slope. This response is the closest comparison to Ableton Live's EQ Eight, but this plug-in does not include parametric ability.



This is the response of the Waves REQ 2 which also looks similar to Ableton Live's EQ Eight and Waves VEQ 3. My understanding for the two different types of phase shifting is the difference in circuitry. Overall, neither of the phase responses result in better sound because there is still a shift in phase in the effected frequency range.

When starting this project, I was mainly focused on creating the parametric ability for an EQ, but with my research and curiosity, I learned the difference between linear and non-linear phase EQ signal processing. I created an eight band parametric non-linear phase EQ, which is comprable to the EQ portion of iZotope's Neutron 3. Future improvements to this project may be to not use the ProcessorChain. It made it easier to keep track of the filters, but to get a phase response comprable to Ableton Live's EQ Eight, I would have to use the StateVariableTPTFilter instead of the IIR filter for low and highpass filters. This trading between filter objects to create parametric ability makes it more complex to implement inside the ProcessorChain. I am also planning on exploring the FIR filter class, and code a linear phase EQ in the near future. Even though there are just a few rare instances where linear phase EQs are needed, learning about them and testing the phase responses of the plug-ins that I own has made me more cognisant when mixing and gave me a better understanding of which EQs to use for certain reasons.

If you are interested in looking at the code, you can check it out on my GitHub here. I am planning on creating more consistent audio development projects. Exploring more complex audio processing concepts, user friendly interfaces, and even new developmental tools like AI.